

Phase 1F — Website Builder Module & the Reusable Module Pattern

Phase 1F delivers two things of equal weight:

1. **The Website Builder module** — the free module every tenant gets, letting a tenant create, edit, publish, and preview simple websites.
2. **A reusable module pattern** — the convention every future BaseCenter module follows for how it renders inside the tenant workspace, stores its own data, and respects subscription/seat gating.

Both are built on the Phase 1A–1E foundation (JWT auth, tenant portal, modules, subscriptions) with no duplication. The Website Builder is the reference implementation of the pattern.

Part A — The Reusable Module Pattern

A module is a self-contained slice of the product. The pattern has four pillars:

1. Module-owned, tenant-scoped data

- Each module keeps **its own tables**. Website Builder adds `websites` and `website_blocks`.
- Every row carries the owning `tenant_id` (indexed). Data is never shared across tenants.

2. Service layer enforces tenant isolation

- All tenant-scoped queries **filter by** `tenant_id` (`website_service.py`).
- Fetching a resource that exists but belongs to another tenant returns **404** (existence is never leaked), not 403.

3. A reusable access guard — `require_active_module(slug)`

- `backend/app/api/module_guard.py` exposes `require_active_module(module_slug)`, a factory that returns a FastAPI dependency which:
 1. resolves the authenticated tenant user (`get_current_tenant_user`),
 2. loads the owning `Tenant`,
 3. verifies an **active** `Subscription` join `Module` for that slug,
 4. raises **HTTP 403** if the module is not active for the tenant,
 5. returns the `Tenant` for downstream tenant-scoped queries.
- Any module router adds gating + a tenant handle with a single dependency and **zero per-module boilerplate**:

```
```python
_guard = require_active_module("website-builder")
```

```
@router.get("/websites")
```

```
def list_websites(tenant: Tenant = Depends(_guard), db: Session = Depends(get_db)):
```

```
return website_service.list_websites(db, tenant)
...
```

## 4. Dynamic UI convention

- A module's UI lives under `/app/modules/<slug>` in the tenant portal.
- `TenantShell` holds a **MODULE\_NAV registry keyed by module slug**. It fetches the tenant's active modules (`getTenantDashboard().active_modules`) and renders a nav entry **only for modules that are active**. Adding a future module to the registry makes it appear automatically once the tenant activates it.

**To add a new module later:** create its models (with `tenant_id`), a service that filters by `tenant_id`, a router guarded by `require_active_module("<slug>")` mounted under `/api/v1/tenant/<slug>`, a UI area under `/app/modules/<slug>`, and one entry in `MODULE_NAV`.

# Part B — Website Builder Feature

## 1. Data model

- **Website** — `id`, `tenant_id` (indexed FK), `name`, `slug` (globally unique, drives the public route), `published` (bool, default false), `settings` (JSON for branding/meta), timestamps. `blocks` relationship cascades on delete.
- **WebsiteBlock** — `id`, `website_id` (FK), `block_type` (`heading` | `text` | `image` | `button`), `content` (structured JSON, shape depends on type), `position` (int), timestamps.

## 2. Backend API (all tenant endpoints gated by the module guard)

Mounted at `/api/v1/tenant/website-builder`:

- `GET /websites` · `POST /websites` · `GET/PUT/DELETE /websites/{id}`
- `POST /websites/{id}/publish` — toggle published state
- `POST /websites/{id}/blocks` · `PUT/DELETE /websites/{id}/blocks/{blockId}`
- `POST /websites/{id}/blocks/reorder` — reorder blocks by id + position

Public rendering at `/api/v1/public` (no auth):

- `GET /sites/{slug}` — returns a **published** site's blocks only; unpublished or unknown slugs return **404**.

## 3. Editor UI (tenant portal)

- `/app/modules/website-builder` — list all sites, create a site, open the editor, view a published site, delete.
- `/app/modules/website-builder/[id]` — full editor: edit name/slug, save settings, **publish/unpublish** toggle, add blocks (`heading/text/image/button`), edit block content inline, **reorder** blocks (up/down), delete blocks, and a "View live" link.

## 4. Public preview

- `/site/{slug}` — standalone, read-only render of a published site's blocks. No tenant shell, no auth, and the marketing nav/footer are suppressed on `/site/*`. Unpublished/unknown sites show a friendly "Site not available".

## Files

### Backend

- `app/models/website.py` — `Website` + `WebsiteBlock` models (registered in `app/models/__init__.py` and `alembic/env.py`).
- `alembic/versions/d2b3c4e5f6a7_phase1f_website_builder.py` — creates `websites` + `website_blocks` (down\_revision `c1a2b3d4e5f6`).
- `app/api/module_guard.py` — `require_active_module` (reusable guard).
- `app/schemas/website.py` — website/block/public schemas.
- `app/services/website_service.py` — tenant-scoped CRUD, reorder, publish, public fetch.
- `app/api/v1/website_builder.py` — guarded tenant router.
- `app/api/v1/public.py` — public (no-auth) render router.
- `app/main.py` — mounts both routers.

### Frontend

- `app/app/app/modules/website-builder/page.tsx` — site list + create.
- `app/app/app/modules/website-builder/[id]/page.tsx` — editor.
- `app/app/site/[slug]/page.tsx` — public preview.
- `components/tenant/TenantShell.tsx` — `MODULE_NAV` registry + dynamic module nav.
- `components/layout/Navigation.tsx`, `Footer.tsx` — suppressed on `/site/*`.
- `lib/api.ts` — Website Builder types + client functions (tenant + public).

## Verification

- PostgreSQL running; `alembic upgrade head` applies cleanly (head `d2b3c4e5f6a7`); `websites` and `website_blocks` tables created.
- Backend ( :8000 ) imports and registers all `/api/v1/tenant/website-builder/*` and `/api/v1/public/*` routes.
- `npm tsc --noEmit` and `npm run build` both pass; new routes build (`/app/modules/website-builder`, `/app/modules/website-builder/[id]`, `/site/[slug]`).
- End-to-end smoke test (API + browser):
- Tenant signup **auto-activates** the free Website Builder module; the nav shows “Website Builder” dynamically.
- Create site → add heading + text blocks → **reorder** (verified new order) → publish → `GET /public/sites/{slug}` returns 200 with blocks.
- **Unpublished** site → public preview returns **404** (not viewable).
- **Module guard**: with the tenant’s website-builder subscription set to `cancelled`, the endpoint returns **403**; active again → 200.
- **Tenant isolation**: Tenant B requesting Tenant A’s website id returns **404**; Tenant B’s list is empty.
- Invalid `block_type` returns **400**.

- Browser: public `/site/{slug}` renders read-only with no marketing chrome; editor list and detail pages render and operate.

---

## Reusable Module Pattern Summary

---

Pillar	Mechanism
Data ownership	Module-owned tables, every row scoped by <code>tenant_id</code>
Isolation	Service layer filters by <code>tenant_id</code> ; cross-tenant → 404
Access gating	<code>require_active_module(slug)</code> dependency → 403 if inactive
API mounting	Router under <code>/api/v1/tenant/&lt;slug&gt;</code>
UI	<code>/app/modules/&lt;slug&gt;</code> + <code>MODULE_NAV</code> registry (active-only nav)
Public render	<code>/api/v1/public/*</code> + <code>/site/*</code> (published only, no auth)